

TEAM8

OCTOBER 6



LEXER

INDEX



03 INTRODUCTION

04 THE PROBLEM

05 SOLUTION

09 BENEFITS

10 CONCLUSION

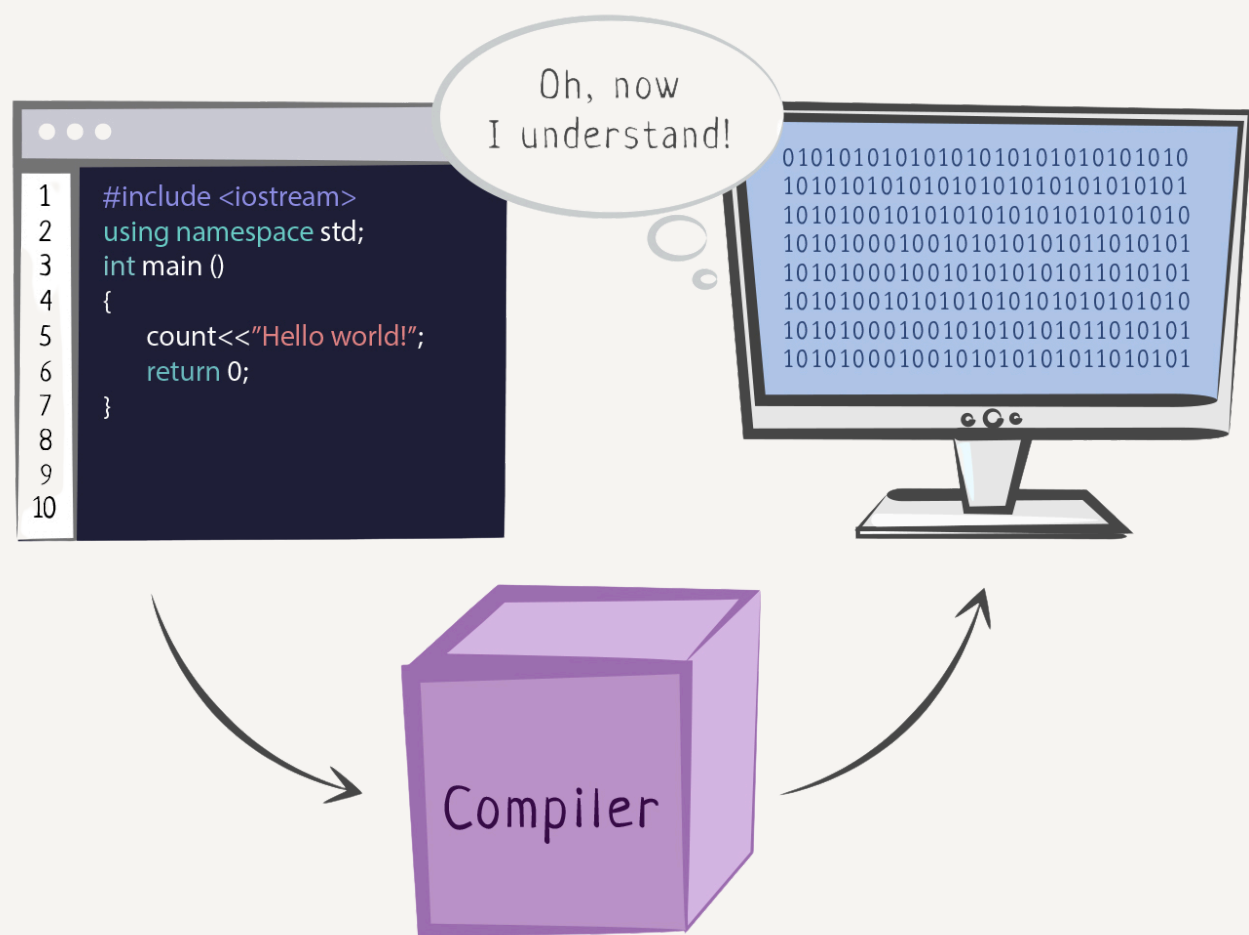
INTRODUCTION*

Daily as a programmer, we're used to compile, see the output of the compiler and then recompile a lot of times.

This process is ignored and just treated as an abstract part of the programming steps. Lets see how it works



AS SIMPLE AS POSSIBLE



A compiler turns the high-level code into a language that computers can understand and execute.

Between the phases of the compilation the lexer is a crucial part.

LEXER

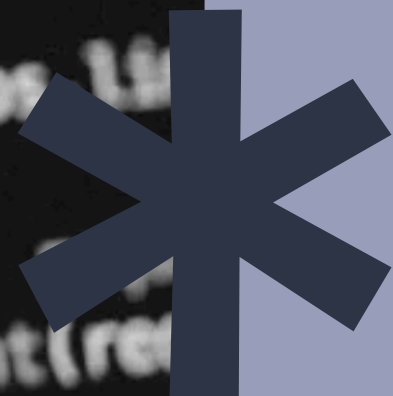
Lexer or lexical analyzer is a program or tool that turns raw characters like code into classified chunks known as tokens.

```
self.file = open(os.path.join(job_dir, 'fingerprint.log'), 'a')
self.fingerprints = set()
self.logdupes = True
self.debug = debug
self.logger = logging.getLogger(__name__)
if path:
    self.file = open(os.path.join(job_dir, 'fingerprint.log'), 'a')
    self.file.seek(0)
    self.fingerprints.update(set(self.logger.debug('fingerprint.log')))

@classmethod
def from_settings(cls, settings):
    debug = settings.getbool('debug')
    return cls(job_dir(settings), debug)

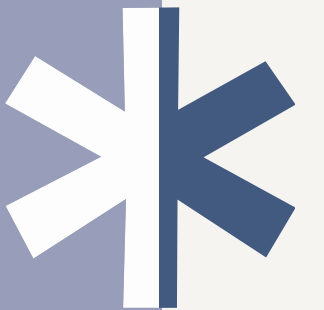
def request_seen(self, request):
    fp = self.request_fingerprint(request)
    if fp in self.fingerprints:
        return True
    self.fingerprints.add(fp)
    if self.file:
        self.file.write(fp + os.linesep)

def request_fingerprint(self, request):
    return request_fingerprint(request)
```



The lexer groups the characters into discrete units known as lexemes, scan them, analyze and classify into different kinds of tokens.

The lexical analyzer is the first step in the compilation process, so it is very important to have a good lexer.



AN EXAMPLE

```
printf("This is an example");  
int a = 10;
```

printf	(	"This is an example"	)	;
identifier	punctuator	literal	punctuator	punctuator
int	a	10	;	
keyword	identifier	constant	punctuator	

token count: 9

AN EXAMPLE

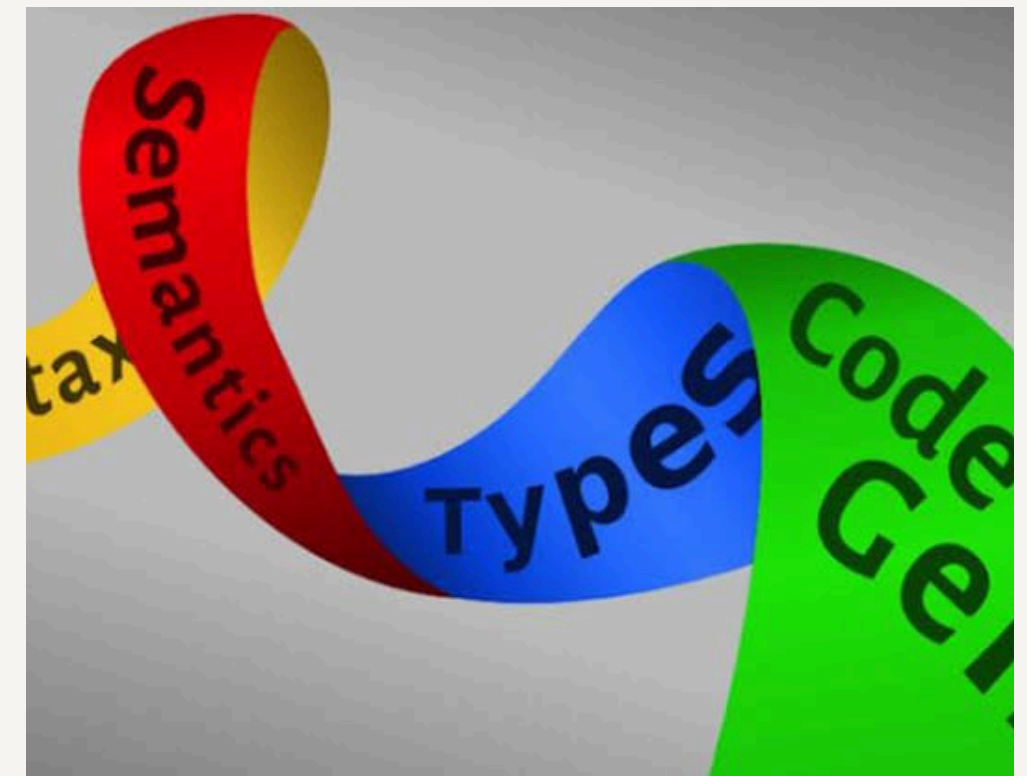
Introducing the same string into our lexer the given output is

```
<TOKEN_IDENTIFIER, NO_SUBTYPE_VALUE, printf>
<TOKEN_PUNCTUATION, PUNCTUATION_LPAREN, (>
<TOKEN_LITERAL, NO_SUBTYPE_VALUE, "This is an example">
<TOKEN_PUNCTUATION, PUNCTUATION_RPAREN, )>
<TOKEN_PUNCTUATION, PUNCTUATION_SEMICOLON, ;>
<TOKEN_KEYWORD, KEYWORD_INT, int>
<TOKEN_IDENTIFIER, NO_SUBTYPE_VALUE, a>
<TOKEN_OPERATOR, OPERATOR_ASSIGN, =>
<TOKEN_CONSTANT, NO_SUBTYPE_VALUE, 10>
<TOKEN_PUNCTUATION, PUNCTUATION_SEMICOLON, ;>
```

```
Token counters:
TOKEN_KEYWORD: 1
TOKEN_IDENTIFIER: 2
TOKEN_PUNCTUATION: 4
TOKEN_OPERATOR: 1
TOKEN_CONSTANT: 1
TOKEN_LITERAL: 1
TOTAL_VALID_TOKENS: 10
```

BENEFITS

A good compiler is the foundation of a good language. Our lexer gives you precise tokens, early error detection, and optimal performance.



AS CONCLUSION



The lexer is the first step in compilation. It deserves the importance our team gives it: a robust, fast, and easy-to-use solution.

OUR MEMBERS

MURAT CALDERON (PRESENTER)

FRANCISCO REYES (CODER)

DAN CRUZ (PROJECT MANAGER)

VICTOR BRENA (DESIGNER)

CHRISTOPHER DE LA O (WRITER)

